SIMATS ENGINEERING

ASSESSMENT TOOL 3

Name: Mohamed Haroon P
Reg. No: 192511313
Marks Obtained: 95

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA4304**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Code along exercise: Make your first HTML page together with CSS

Weightage: 10%

QUESTIONS: **Total Marks: 10**

Scenario: Web Development Standards Implementation

A start up is developing a modern web application. During development, the team encounters multiple issues related to HTML elements, HTTP methods, XML syntax, and web communication protocols. The following observations are made:

- The team needs to embed a promotional video on the homepage.
- A registration form must securely send user data to the server.
- The navigation menu must follow semantic HTML standards.
- XML configuration files are being used but contain syntax errors.
- The application must communicate over the correct web protocol.

Data Snippets Provided

Snippet 1: Video Embedding Options

A. <media src="video.mp4"></media>
B. <video src="video.mp4" controls></video>
C. <movie src="video.mp4"></movie>
D. <vid src="video.mp4"></vid>

Snippet 2: Form Submission Methods

A. GET
B. POST
C. PUT
D. DELETE

Snippet 3: Navigation Elements

A.
B. <nav>
C. <section>
D. <header>

Snippet 4: XML Syntax

A. <note><to>User</to><from>Admin</from>
B. <note><to>User</to><from>Admin</from></note>
C. <note><to>User</from></note>
D. <note to="User"></note

Snippet 5: Web Communication Protocols

A. FTP
B. HTTP C. SMTP
D. SNMP

Questions:

1. Select the **correct HTML tag for video embedding** and justify your answer.
2. Identify the **appropriate HTTP method for form submission** and explain why.
3. Choose the **proper semantic element for navigation** and justify its usage.
4. Identify the **correct XML syntax** and explain the error in other options.
5. Choose the **correct protocol for web communication** and justify your choice.

Code of Conduct:

I **Mohamed Haroon P (192511313)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	24 / 25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	24 / 25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	19.2 / 20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	19.2 / 20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	9.4 / 10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

SCENARIO & QUICK ANSWER KEY

Scenario: *A start-up is building a web application and must: embed a promotional video, send registration data to the server securely, follow semantic HTML for the navigation menu, fix XML configuration files with syntax errors, and communicate over the correct web protocol. Each of the 5 questions carries 2 marks.*

Q	Snippet	Correct Option	One-Line Reason
1	Video embedding	B <video src="video.mp4" controls></video>	<video> is the only real HTML5 element for video; <media>, <movie> and <vid> do not exist.
2	Form method	B POST	POST sends data in the request body, not in the URL.
3	Navigation element	B <nav>	<nav> is the semantic element for major navigation links.
4	XML syntax	B <note><to>User</to><from>Admin</from></note>	The only well-formed snippet: every tag is closed and properly nested.
5	Web protocol	B HTTP (HTTPS in production)	HTTP is the application-layer protocol that delivers web content.

How to read this solution: every question gives the correct option, a table that evaluates all four options, a worked code example, and a justification. Where a claim can be tested, it was tested: the XML snippets were fed to a real XML parser, and the GET/POST requests were captured from a live test server. The final section combines all five answers into one working HTML page (index.html, delivered with this document).

QUESTION 1 — HTML Tag for Video Embedding

Correct answer: Option B — <video src="video.mp4" controls></video>

Evaluating Every Option

Option	Code	Exists in HTML?	Verdict
A	<media src="video.mp4"></media>	No — there is no <media> element	✗ Browser treats it as an unknown element; nothing plays
B	<video src="video.mp4" controls></video>	Yes — HTML5 video element	✓ Correct
C	<movie src="video.mp4"></movie>	No — there is no <movie> element	✗ Unknown element; nothing plays
D	<vid src="video.mp4"></vid>	No — there is no <vid> element	✗ Unknown element; nothing plays

Justification

- HTML5 introduced <video> (and <audio>) so that browsers can play media natively, without plug-ins such as Flash. It is the only element in the HTML standard designed to embed video.
- The src attribute gives the URL of the video file. The controls attribute is a boolean attribute that makes the browser show its built-in play/pause button, seek bar, volume control and full-screen button. Without controls (and without autoplay or JavaScript), the visitor sees a video that cannot be started.
- <media>, <movie> and <vid> are not defined by the HTML standard. A browser does not reject them: it parses them as generic unknown elements with no behaviour, so the page loads without error but shows no video. This silent failure is why invalid tags are dangerous, and the W3C validator would flag all three.
- <video> is not a void element, so it needs its closing tag. Any text placed between the tags is fallback content, shown only by browsers that cannot play video.

Useful <video> Attributes

Attribute	Purpose
src	URL of the video file (or use <source> children for several formats).
controls	Shows the browser's built-in playback controls.
width / height	Reserves space so the page layout does not jump while the video loads.
poster	Image shown before the video starts playing.
preload	Hints how much to download in advance: none, metadata or auto.
autoplay + muted	Starts playback automatically. Most browsers block autoplay unless the video is muted.
loop	Restarts the video when it ends.

Minimal Correct Form (Option B)

```
<video src="video.mp4" controls></video>
```

Production-Quality Version for the Start-Up's Homepage

```
<video controls width="640" height="360" poster="poster.jpg" preload="metadata">
  <source src="video.mp4"  type="video/mp4">
  <source src="video.webm" type="video/webm">
  <track kind="captions" src="captions.vtt" srclang="en" label="English">
  Your browser does not support the video tag.
  <a href="video.mp4">Download the video</a>.
</video>
```

The <source> elements let the browser pick the first format it can play, <track> adds captions for deaf or hard-of-hearing users, and the fallback text and link keep the content reachable on very old browsers.

✓ Key Takeaway: Option B, <video src="video.mp4" controls></video>, is correct because

<video> is the HTML5 element for embedding video and controls provides the play/pause interface. <media>, <movie> and <vid> are not HTML elements, so browsers ignore them and the video never plays.

QUESTION 2 — HTTP Method for Secure Form Submission

Correct answer: Option B — POST

Evaluating Every Option

Option	Method	Intended Purpose	Where Data Travels	Idempotent?	Verdict for Registration
A	GET	Retrieve a resource (read-only)	In the URL query string	Yes (and safe)	✗ Exposes user data in the URL
B	POST	Submit data to be processed, e.g. create a new account	In the request body	No	✓ Correct
C	PUT	Replace or create a resource at a URL the client already knows	In the request body	Yes	✗ Wrong meaning; HTML forms cannot send PUT
D	DELETE	Remove a resource	Usually no body	Yes	✗ Would delete, not register; HTML forms cannot send DELETE

HTML's form method attribute accepts only get and post (plus dialog). PUT and DELETE are available from JavaScript (fetch/XMLHttpRequest) or through server-side method override, not from a plain <form>.

Verified Evidence: The Same Data Sent With GET and With POST

A small test server was started and the same registration data was sent once with GET and once with POST. The server printed exactly the bytes it received:

Raw request received — GET (password visible in the first line)

```
GET /register?name=Asha+Kumar&password=Secret123 HTTP/1.1
Accept-Encoding: identity
Host: app.example.com
User-Agent: demo-browser/1.0
Accept: text/html
```

Raw request received — POST (password only in the body)

```
POST /register HTTP/1.1
Accept-Encoding: identity
Content-Length: 34
Host: app.example.com
User-Agent: demo-browser/1.0
Accept: text/html
Content-Type: application/x-www-form-urlencoded

name=Asha+Kumar&password=Secret123
```

- With GET, the password is part of the request line (GET /register?name=...&password=Secret123). That text is stored in browser history and bookmarks, written to web-server and proxy logs, and can leak through the Referer header.
- With POST, the request line is only POST /register. The data sits in the body, described by ContentType: application/x-www-form-urlencoded and Content-Length: 34 (the body really is 34 bytes long).
- POST has no practical size limit for form data, whereas URLs are limited in length. POST also is not repeated silently on refresh or Back, because browsers warn before resubmitting.

Why POST Is Right, and What 'Securely' Still Requires

Registration changes server state (it creates an account), which is what POST is for. GET must be safe and read-only, so it should never be used for actions that create or change data.

Important: POST keeps data out of the URL, but it does not encrypt it. To be truly secure, the form must also be served over HTTPS, the server must validate input and hash passwords, and a CSRF token should protect the form.

Correct Form

```
<form action="/register" method="POST">
  <label for="name">Full name</label>
  <input type="text" id="name" name="name" required>
```

```
  <label for="password">Password</label>
  <input type="password" id="password" name="password" required minlength="8">

  <button type="submit">Register</button>
</form>
```

✓ **Key Takeaway:** **Option B, POST, is correct: it carries the registration data in the request body instead of the URL, is meant for creating or changing server data, and is supported by HTML forms. GET leaks data in the URL; PUT and DELETE have different meanings and cannot be sent by a plain HTML form. Security still needs HTTPS on top of POST.**

Correct answer: Option B — <nav>

Evaluating Every Option

Option	Element	What It Means	Verdict for a Menu
A		A generic container with no meaning at all	✗ Gives no information to browsers or screen readers
B	<nav>	A section containing the page's major navigation links	✓ Correct
C	<section>	A thematic grouping of content, normally with its own heading	✗ A menu is not a standalone topic of content
D	<header>	Introductory content or the page banner; it may contain a <nav>	✗ It wraps the menu but is not itself the navigation

Justification

- Accessibility: <nav> is exposed to assistive technology as a navigation landmark, so a screen-reader user can jump straight to the menu or skip past it. A offers no such shortcut.
- Semantics and SEO: the element states what the content is, so search engines and other tools can tell the site's main links from ordinary content.
- Standards compliance and maintenance: semantic elements make the document outline selfexplanatory, and they give CSS and scripts a meaningful hook (nav a { ... }) instead of class names such as div.menu.
- Correct placement: the menu is normally a list of links, so it goes inside the <header> as <nav>
<a>. Reserve <nav> for major blocks such as the main menu or a table of contents, not for every small group of links.

Semantic Navigation (HTML)

```
<header>
  <h1>NovaStart</h1>
  <nav aria-label="Main navigation">
```

—

```
  <ul>
    <li><a href="#top" aria-current="page">Home</a></li>
    <li><a href="#promo">Promo Video</a></li>
    <li><a href="#register">Register</a></li>
  </ul>
 </nav>
</header>
```

Styling the Menu With CSS (turns the list into a horizontal menu)

```
nav ul   { list-style: none; margin: 0; padding: 0; display: flex; gap: 20px; }
nav a    { color: #ffffff; text-decoration: none; font-weight: bold; } nav
a:hover, nav a:focus { text-decoration: underline; } nav a[aria-
current="page"] { border-bottom: 3px solid #c00000; }
```

aria-label names the landmark (useful when a page has more than one <nav>), and aria-current="page" tells assistive technology which link is the current page.

✓ Key Takeaway: Option B, <nav>, is correct: it is the HTML5 semantic element for major navigation links, giving screen-reader users a navigation landmark and giving search engines a clear page structure. has no meaning, <section> groups themed content, and <header> is a banner container that may hold the <nav>.

Correct answer: Option B — <note><to>User</to><from>Admin</from></note>

Verified Result: Each Snippet Run Through a Real XML Parser

Each snippet was parsed with Python's xml.etree.ElementTree (the Expat parser). The error messages below are the parser's own output, not paraphrased:

Option	Snippet	Parser Result	Explanation of the Error
A	<note><to>User</to><from>Admin</from>	no element found: line 1, column 37	The root element <note> is opened but never closed. The closing tag </note> is missing, so the document ends too early.
B	<note><to>User</to><from>Admin</from></note>	parsed OK, root = <note>, children = ['to', 'from']	No error: one root, every tag closed, correct nesting. ✓ Well-formed.
C	<note><to>User</from></note>	mismatched tag: line 1, column 16	Mismatched tags: <to> is closed by </from>. An end tag must match the most recently opened start tag, so the nesting is wrong (and <from> was never opened).
D	<note to="User"></note	unclosed token: line 1, column 16	The end tag </note is missing its closing >, so the parser reaches the end of the text inside an unfinished token.

The Rules That Make XML Well-Formed

Rule	Meaning	Broken By
Single root element	The whole document sits inside exactly one outermost element.	—
Every start tag has a matching end tag	<tag> ... </tag>, or a self-closing <tag/>.	A (missing </note>), D (unfinished end tag)

—

Proper nesting	Elements close in the reverse order they were opened.	C (<to> closed by </from>)
Case sensitivity	<Note> and <note> are different names.	—
Quoted attribute values	to="User", never to=User.	—
Escaped special characters	Use < & > inside text.	—

Note the difference between well-formed (obeys these syntax rules, which every XML parser requires) and valid (also obeys a DTD or XSD schema). Option B is well-formed; checking validity would need a schema.

Corrected Versions of the Broken Snippets (all verified to parse)

```
<!-- A, fixed: add the missing closing tag -->
<note><to>User</to><from>Admin</from></note>

<!-- C, fixed: close <to> with </to> -->
<note><to>User</to></note>

<!-- D, fixed: complete the end tag -->
<note to="User"></note>
```

✓ Key Takeaway: Option B is the only well-formed XML: it has one root <note>, and <to> and <from> are opened and closed in order. A lacks </note>, C closes <to> with </from> (improper nesting), and D has an unfinished end tag </note. The real parser confirmed all four results.

Correct answer: Option B — HTTP (in production, HTTPS = HTTP over TLS)

Evaluating Every Option

Option	Protocol	Full Form and Purpose	Default Port*	Fit for Web Pages?
A	FTP	File Transfer Protocol — moves files between a client and a server	21 (TCP)	✗ Not used to deliver web pages to browsers
B	HTTP	HyperText Transfer Protocol — request/response protocol that delivers web pages, images, APIs and other web content	80 (TCP); HTTPS 443 (TCP)	✓ Correct
C	SMTP	Simple Mail Transfer Protocol — sends and relays e-mail between mail servers	25 (TCP); submission 587 (TCP)	✗ E-mail, not web content
D	SNMP	Simple Network Management Protocol — monitors and manages network devices	161 (UDP)	✗ Network administration, not web content

* Port numbers were read from the operating system's service database (they are the standard registered ports).

Justification

- HTTP is the application-layer protocol of the World Wide Web. Every http:// or https:// address you type in a browser means "fetch this resource using HTTP".
- It works as request → response: the browser sends a request (method, path, headers, optional body) and the server answers with a status code, headers and the content (HTML, CSS, JavaScript, images, XML or JSON).
- It defines exactly the features the case study needs: methods such as GET and POST (Question 2), status codes (200 OK, 404 Not Found), and headers such as Content-Type that tell the browser how to treat HTML, CSS or XML.
- It normally runs over TCP (HTTP/1.1 and HTTP/2); HTTP/3 runs over QUIC, which is built on UDP. Either way, it is the protocol browsers use for web pages.

Verified Request–Response Cycle

The POST request captured in Question 2 received this response from the test server:

Request (browser → server)

—

```
POST /register HTTP/1.1
Accept-Encoding: identity
Content-Length: 34
Host: app.example.com
User-Agent: demo-browser/1.0
Accept: text/html
Content-Type: application/x-www-form-urlencoded

name=Asha+Kumar&password=Secret123
```

Response (server → browser)

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
Connection: close

OK
```

Use HTTPS, Not Plain HTTP, in Production

- Encryption: TLS scrambles the traffic, so passwords from the registration form cannot be read on the network.
- Integrity: the content cannot be altered in transit without detection.
- Authentication: the browser can confirm it is really talking to the start-up's server.
- Browsers label plain-HTTP pages "Not secure" and restrict several modern features to HTTPS pages.

✓ Key Takeaway: Option B, HTTP, is correct: it is the protocol browsers and web servers use to exchange web pages and form data. FTP transfers files, SMTP sends e-mail, and SNMP manages

network devices. In production the start-up should use HTTPS (HTTP over TLS) so that registration data is encrypted.

CODE-ALONG: YOUR FIRST HTML PAGE WITH CSS

index.html (HTML + CSS in one file)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NovaStart - Home</title>
  <style>
    * { box-sizing: border-box; }     body { margin: 0; font-family: Arial, Helvetica, sans-serif; color:
#1a1a1a; background: #ffffff; lineheight: 1.5; }

    /* ---------- header + semantic navigation ---------- */     header { background: #1a1a1a; color: #ffffff;
padding: 16px 24px; border-bottom: 4px solid #c00000;
}     header h1 { margin: 0 0 8px; font-size: 1.6rem; }     nav ul { list-style: none; margin: 0;
padding: 0; display: flex; gap: 20px; flex-wrap: wrap; }     nav a { color: #ffffff; text-
decoration: none; font-weight: bold; padding: 4px 2px; }     nav a:hover, nav a:focus { color:
#ff9a9a; text-decoration: underline; }     nav a[aria-current="page"] { border-bottom: 3px
solid #c00000; }

    /* ---------- page body ---------- */
    main { max-width: 760px; margin: 0 auto; padding: 24px; }     section { margin-
bottom: 36px; }     h2 { color: #c00000; border-bottom: 2px solid #c00000;
padding-bottom: 4px; }     video { display: block; width: 100%; max-width: 640px;
background: #000000; }

    /* ---------- registration form ---------- */
    form { display: grid; gap: 8px; max-width: 420px; }
label { font-weight: bold; }
```

```
    input { padding: 8px; border: 2px solid #1a1a1a; border-radius: 4px; font-size: 1rem; }     input:focus
{ outline: 3px solid #c00000; }     button { padding: 10px; background: #c00000; color: #ffffff; border:
0; border-radius: 4px; font-size:
1rem; font-weight: bold; cursor: pointer; }
button:hover { background: #8b0000; }

    footer { background: #1a1a1a; color: #ffffff; text-align: center; padding: 12px; }
</style>
</head>
<body>
  <header id="top">
    <h1>NovaStart</h1>
    <nav aria-label="Main navigation">
      <ul>
        <li><a href="#top" aria-current="page">Home</a></li>
        <li><a href="#promo">Promo Video</a></li>
        <li><a href="#register">Register</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="promo">
      <h2>See NovaStart in 60 seconds</h2>
      <!-- Snippet 1, option B: the HTML5 <video> element with the controls attribute -->
<video controls width="640">
        <source src="video.mp4" type="video/mp4">
Your browser does not support the video tag.
        <a href="video.mp4">Download the video</a>.
      </video>
    </section>

    <section id="register">
```

```
<h2>Create your account</h2>
```

```
    <!-- Snippet 2, option B: POST sends the data in the request body, not in the URL -->
    <form action="/register" method="POST">
      <label for="name">Full name</label>
      <input type="text" id="name" name="name" required autocomplete="name">

      <label for="email">Email</label>
      <input type="email" id="email" name="email" required autocomplete="email">

      <label for="password">Password</label>
      <input type="password" id="password" name="password" required minlength="8"
autocomplete="new-password">

      <button type="submit">Register</button>
    </form>
  </section>
 </main>

 <footer>
  <p>&copy; 2026 NovaStart</p>
 </footer>
</body>
</html>
```

config.xml (the corrected XML configuration — verified to parse)

```
<?xml version="1.0" encoding="UTF-8"?>
<config>
 <server host="localhost" port="8080"/>
 <protocol>HTTPS</protocol>
 <upload max-size-mb="50"/>
</config>
```

How the Page Meets Each Requirement of the Scenario

Requirement in the Scenario	Where It Is Met
Embed a promotional video	<video controls width="640"> with a <source> and fallback text (Question 1).
Securely send registration data	<form action="/register" method="POST"> with required fields; deploy over HTTPS (Questions 2 and 5).
Semantic navigation menu	<nav aria-label="Main navigation"> inside <header>, styled horizontally with CSS flexbox (Question 3).
Fix the XML configuration files	config.xml is well-formed: one root <config>, every tag closed (Question 4).
Communicate over the correct protocol	<protocol>HTTPS</protocol> in the configuration; the page is requested and the form is submitted over HTTP/HTTPS (Question 5).